

FACULTAD DE INGENIERÍA Y TECNOLOGÍA
Ingeniería en Sistemas de Información
Paradigmas y Lenguajes de Programación II

ACTIVIDAD DE EVALUACIÓN 2

Simulador de Procesos y Memoria

Informe técnico – Programación Orientada a Objetos

Estudiante: Sophia Saquer

DNI: 46857995

Sede: Corrientes

Docentes: José A. Fernandez – S. Romina Stacciuoli

Año: 2026

Fecha de entrega: 05/10/2026

INTRODUCCIÓN

El presente informe técnico corresponde a la Actividad de Evaluación 2 de la asignatura Paradigmas y Lenguajes de Programación II, desarrollada de manera articulada con Sistemas Operativos. El trabajo tiene como eje la construcción de un simulador de procesos y memoria aplicando principios de Programación Orientada a Objetos, con el objetivo de representar de forma clara y verificable el comportamiento de distintos componentes propios de un sistema operativo.

La solución fue implementada en TypeScript como una biblioteca de clases, sin interfaz gráfica ni programa principal de demostración. Su funcionamiento se valida mediante pruebas automatizadas, lo que permite comprobar aspectos como la gestión de procesos, la asignación y liberación de memoria, la planificación de CPU mediante Round Robin, los eventos de Entrada/Salida, las métricas del sistema y el cumplimiento de los requerimientos funcionales definidos para la actividad.

El desarrollo también pone énfasis en el diseño orientado a objetos, especialmente en la distribución de responsabilidades, el encapsulamiento del estado interno, el uso de interfaces, la composición entre objetos y la construcción de un modelo coherente con los requerimientos planteados. La propuesta busca que cada componente tenga una responsabilidad clara y que el comportamiento general del simulador pueda ser comprendido, probado y defendido de manera individual.

OBJETIVO Y ESCENARIO DEL SIMULADOR

Objetivo

El objetivo principal del proyecto es aplicar los conceptos de Programación Orientada a Objetos para transformar los requerimientos de un simulador de Sistemas Operativos en un modelo de clases cohesivo, encapsulado y verificable. La actividad busca que el diseño pueda justificarse a partir de las responsabilidades de cada clase, sus relaciones, las abstracciones utilizadas y el comportamiento definido mediante pruebas automatizadas.

En este sentido, la solución desarrollada pretende representar la gestión de procesos, la asignación de memoria y la planificación de CPU mediante una estructura de clases que permita aplicar principios como abstracción, encapsulamiento, composición, uso de interfaces y polimorfismo. Además, el funcionamiento debe poder verificarse mediante tests unitarios y de integración, manteniendo una cobertura de código superior al 90 %.

Escenario del simulador

El escenario plantea un laboratorio universitario que necesita un motor de simulación capaz de representar cómo distintos procesos comparten una memoria limitada y una única CPU. Cada proceso solicita una cantidad determinada de memoria, espera su turno para utilizar el procesador, consume unidades de CPU, puede bloquearse temporalmente por operaciones de Entrada/Salida y libera sus recursos cuando finaliza.

La simulación comienza en el tick 0, con la memoria completamente libre y la CPU sin procesos en ejecución. Tanto el tamaño total de la memoria como el quantum del planificador son configurables; como valores de referencia, la consigna propone 1024 KB de memoria y un quantum de 2 ticks.

El motor avanza de manera discreta mediante ticks, evitando depender del tiempo real. Durante cada tick se realiza el intento de admisión de procesos, se actualizan los procesos bloqueados, se ejecuta la planificación de CPU mediante Round Robin y finalmente se actualiza el reloj junto con las métricas del sistema. Como máximo un proceso puede consumir una unidad de CPU por tick.

Para la gestión de memoria se utiliza asignación contigua con tres políticas intercambiables (First Fit, Best Fit y Worst Fit), seleccionables al crear el simulador, junto con mecanismos de liberación y coalescencia de bloques libres.

DESCRIPCIÓN GENERAL DE LA SOLUCIÓN

La solución desarrollada consiste en una biblioteca de clases implementada en TypeScript que simula la gestión de procesos, memoria principal y planificación de CPU. De acuerdo con la consigna, el proyecto no utiliza interfaz gráfica, menú de consola ni una función main; el comportamiento del sistema se ejecuta y verifica mediante pruebas automatizadas.

El componente central es la clase `Simulador`, encargada de coordinar el funcionamiento general sin concentrar toda la lógica del sistema. Para ello colabora con otros objetos especializados. La clase `Proceso` representa cada proceso y administra su estado, CPU restante, quantum consumido y comportamiento asociado a operaciones de Entrada/Salida. La clase `Memoria` mantiene los bloques de memoria y delega la búsqueda de espacio a una política de asignación. Se implementaron tres políticas: `FirstFit`, `BestFit` y `WorstFit`. La política se elige al crear el `Simulador` y, si no se indica, se utiliza `First Fit`.

La planificación de CPU se encuentra separada en `PlanificadorRoundRobin`, que mantiene la cola de procesos listos respetando un comportamiento FIFO. El simulador utiliza esta cola para despachar procesos y aplicar las reglas de Round Robin según el quantum configurado. Cuando un proceso termina, su memoria es liberada; si aparecen bloques libres contiguos, `Memoria` realiza automáticamente su coalescencia. El sistema también permite representar bloqueos por Entrada/Salida, durante los cuales el proceso conserva la memoria asignada, pero deja temporalmente de utilizar la CPU.

El diseño utiliza interfaces para definir los contratos principales de los distintos componentes, entre ellas `ISimulador`, `IProceso`, `IMemoria`, `IPlanificador` e `IPoliticaAsignacion`. También se definieron interfaces específicas para las métricas y para las vistas de consulta del estado. Estas vistas permiten obtener información del simulador sin entregar referencias modificables a sus objetos internos, protegiendo las reglas del dominio y favoreciendo el encapsulamiento. Esto responde al requerimiento de consultar procesos y estado del sistema mediante vistas de solo lectura o copias seguras.

El avance de la simulación se realiza mediante ticks discretos y deterministas. En cada tick se intenta admitir procesos y asignar memoria, se actualizan los procesos bloqueados, se realiza el despacho y ejecución de CPU y finalmente se actualiza el reloj junto con las métricas correspondientes. De esta forma, la simulación puede reproducirse siempre con los mismos resultados para un mismo escenario de prueba.

Finalmente, el funcionamiento de la solución se valida mediante Vitest, utilizando pruebas unitarias y de integración para verificar los requerimientos funcionales, los casos límite y las situaciones de error. Además, el repositorio utiliza GitHub Actions para ejecutar automáticamente la verificación de TypeScript, los tests y la cobertura de código en cada actualización del proyecto, permitiendo mantener una solución reproducible y verificable.

DISEÑO Y RESPONSABILIDADES DE LAS CLASES

El diseño de la solución se organizó distribuyendo el comportamiento del simulador entre distintas clases, evitando concentrar toda la lógica en un único objeto. Cada clase posee una responsabilidad específica dentro del dominio y colabora con las demás para completar el funcionamiento general. Esta separación permite mantener una mayor cohesión y facilita tanto las pruebas como la modificación independiente de cada componente, aspecto que forma parte de los criterios de evaluación de la actividad.

La clase Simulador funciona como coordinadora principal. Es responsable de registrar los procesos, mantener el tick actual, administrar cuál se encuentra utilizando la CPU y coordinar el orden de las operaciones que se realizan en cada tick. Para cumplir esta función se comunica con la memoria, el planificador y los procesos, pero delega en ellos las operaciones específicas. También calcula las métricas generales y construye las vistas utilizadas para consultar el estado del sistema sin exponer directamente sus objetos internos.

La distribución de responsabilidades implementada es la siguiente:

Clase	Responsabilidad principal
<i>Simulador</i>	Coordinar el funcionamiento general, registrar procesos, avanzar los ticks, administrar la CPU, controlar cambios de contexto, coordinar memoria y planificación, y exponer el estado y las métricas del sistema.
<i>Proceso</i>	Representar un proceso y proteger su estado interno. Mantiene CPU restante, estado actual, quantum consumido y tiempo de bloqueo. También controla sus transiciones de estado y los eventos de Entrada/Salida.
<i>Memoria</i>	Administrar los bloques de memoria, asignar y liberar espacio, conservar la estructura ordenada de bloques y realizar la coalescencia de bloques libres adyacentes.
<i>BloqueMemoria</i>	Representar una región contigua de memoria mediante su posición inicial, tamaño y PID asignado. Permite determinar si un bloque se encuentra libre.
<i>FirstFit</i>	Implementar la política de asignación First Fit, buscando el primer bloque libre cuyo tamaño sea suficiente para satisfacer una solicitud.
<i>BestFit</i>	Implementar la política Best Fit, buscando el bloque libre suficiente de menor tamaño; ante empates elige el de menor dirección.
<i>WorstFit</i>	Implementar la política Worst Fit, buscando el bloque libre suficiente de mayor tamaño; ante empates elige el de menor dirección.
<i>PlanificadorRoundRobin</i>	Mantener la cola FIFO de procesos listos, permitiendo encolar y desencolar procesos según el funcionamiento de Round Robin. Para las consultas externas expone únicamente el orden de los PID.
<i>EstadoProceso</i>	Definir los posibles estados que puede adoptar un proceso: Nuevo, Esperando Memoria, Listo, Ejecutando, Bloqueado y Terminado.

Coordinación mediante Simulador

Aunque Simulador es el punto central de la ejecución, no implementa directamente todos los algoritmos. Por ejemplo, cuando un proceso necesita memoria, el simulador solicita a Memoria que intente realizar la asignación. A su vez, Memoria delega la selección del bloque en la política de asignación configurada.

De la misma manera, el simulador no mantiene manualmente el orden de los procesos listos, sino que utiliza PlanificadorRoundRobin. Cuando existe una CPU libre, solicita el siguiente proceso de la cola y posteriormente ordena al propio objeto Proceso que cambie a estado Ejecutando.

Esta organización permite representar las relaciones de colaboración entre objetos sin asignar responsabilidades que no correspondan a cada clase.

Responsabilidad de Proceso

Proceso contiene la información necesaria para representar el ciclo de vida de cada proceso. Sus datos principales son el PID, la memoria requerida y el tiempo total de CPU, además de información que se modifica durante la simulación, como la CPU restante, el estado, el quantum consumido y el tiempo de bloqueo.

Las operaciones que cambian su estado se encuentran controladas por la propia clase. Por ejemplo, un proceso solo puede pasar a Ejecutando cuando previamente se encuentra Listo, y únicamente puede marcarse como Terminado cuando agotó su tiempo de CPU. De esta manera, las reglas de transición pertenecen al objeto que representa el proceso y no quedan dispersas en otras clases.

También es responsable del comportamiento relacionado con Entrada/Salida. El proceso almacena cuándo debe producirse el evento, su duración y si ya fue ejecutado. Cuando se bloquea conserva su memoria y su temporizador se actualiza durante los ticks posteriores, tal como establece el comportamiento requerido para la simulación.

Responsabilidad de Memoria y BloqueMemoria

La clase Memoria representa la memoria principal como una colección ordenada de objetos BloqueMemoria. Inicialmente contiene un único bloque libre que abarca todo el tamaño configurado.

Cuando se solicita una asignación, utiliza la política configurada para localizar un bloque adecuado. Si el bloque posee un tamaño mayor al solicitado, se divide en una parte ocupada y otra libre. Si el tamaño coincide exactamente, se reemplaza directamente por un bloque ocupado.

Cuando un proceso termina, Memoria libera el bloque correspondiente y ejecuta la coalescencia, fusionando los bloques libres contiguos. Esta responsabilidad se encuentra centralizada en la clase para conservar las invariantes de la memoria, como el orden, la continuidad y la ausencia de solapamientos. Estas reglas forman parte de los requerimientos de asignación y liberación de memoria.

BloqueMemoria, por su parte, posee una responsabilidad más simple: representar cada segmento individual mediante su dirección inicial, tamaño y PID. Un PID igual a null representa un bloque libre.

Responsabilidad de las políticas de asignación

FirstFit contiene exclusivamente la estrategia utilizada para buscar un espacio disponible. Recorre los bloques en orden y devuelve la posición del primer bloque libre cuyo tamaño sea mayor o igual al requerido.

La decisión de separar esta lógica de Memoria permite que la memoria se encargue de administrar los bloques mientras que la política determina únicamente qué bloque seleccionar. FirstFit, BestFit y WorstFit implementan la misma interfaz y solo difieren en el criterio de selección.

Responsabilidad de PlanificadorRoundRobin

PlanificadorRoundRobin administra la cola de procesos listos con comportamiento FIFO. Su responsabilidad consiste en agregar procesos al final de la cola, retirar el primero cuando debe ser despachado e informar si existen procesos pendientes.

Las reglas relacionadas con el quantum, los cambios de contexto y la decisión de rotar un proceso son coordinadas por Simulador, mientras que el planificador se limita a administrar correctamente la cola. Además, su consulta del contenido se realiza mediante los PID de los procesos, evitando entregar referencias modificables a los objetos internos.

En conjunto, esta distribución permite que cada clase tenga una función definida y que el simulador se construya mediante la colaboración entre objetos especializados, en lugar de concentrar todo el comportamiento dentro de una única clase.

DECISIONES DE PROGRAMACIÓN ORIENTADA A OBJETOS

El diseño del simulador se realizó buscando aplicar los principios de Programación Orientada a Objetos de forma coherente con el problema planteado. Las principales decisiones estuvieron relacionadas con la distribución de responsabilidades, el encapsulamiento del estado, el uso de interfaces, la composición entre objetos y la separación entre contratos e implementaciones. La consigna evalúa específicamente estos aspectos, además de exigir que el uso o la ausencia de herencia y clases abstractas pueda justificarse.

Separación de responsabilidades

Se decidió dividir el problema en varias clases especializadas en lugar de concentrar toda la lógica en Simulador.

Simulador actúa como coordinador general, mientras que Proceso, Memoria, FirstFit, BestFit, WorstFit, BloqueMemoria y PlanificadorRoundRobin contienen comportamientos específicos de su propio dominio.

Esta decisión favorece la alta cohesión, ya que cada clase concentra operaciones relacionadas entre sí, y reduce el acoplamiento innecesario entre componentes.

Por ejemplo, Simulador no implementa directamente el algoritmo First Fit. Solicita a Memoria que realice la asignación y esta utiliza una política de asignación para seleccionar el bloque correspondiente.

Encapsulamiento del estado

Una de las decisiones más importantes fue mantener privados aquellos atributos cuyo cambio podría alterar las reglas del sistema.

En Proceso, datos como el estado, la CPU restante, el quantum consumido y el tiempo de bloqueo no pueden modificarse directamente desde el exterior. En su lugar se utilizan operaciones como:

- marcarListo()
- marcarEjecutando()
- ejecutarTick()
- marcarTerminado()
- bloquearPorEntradaSalida()

Estas operaciones validan las transiciones permitidas antes de modificar el estado.

De manera similar, Memoria mantiene internamente su colección de bloques y es la propia clase la que controla la asignación, liberación y coalescencia.

El objetivo de esta decisión es impedir que otro componente modifique los datos internos sin respetar las reglas del dominio, criterio especialmente importante dentro de la evaluación de doble encapsulamiento.

Uso de vistas para las consultas

También se decidió no devolver directamente los objetos internos de Simulador.

Para consultar procesos y estado del sistema se utilizan estructuras como IVistaProceso, IVistaBloqueMemoria e IEstadoSistema.

Por ejemplo, cuando se ejecuta:

```
simulador.obtenerProcesos()
```

se reciben representaciones con los datos necesarios para consultar el proceso, pero no el objeto interno real.

Esta decisión evita que un consumidor del simulador pueda modificar directamente un proceso y saltarse las reglas definidas por la clase Proceso. El requerimiento RF02 exige precisamente que los procesos puedan consultarse sin permitir modificaciones externas que eviten las reglas del dominio.

Composición en lugar de una clase única

Se utilizó principalmente composición para construir el simulador.

Conceptualmente, un Simulador:

- tiene una Memoria
- tiene un Planificador
- administra varios Procesos

y una Memoria:

- tiene varios BloqueMemoria
- utiliza una política de asignación

Estas relaciones representan situaciones de tipo “tiene un”, por lo que la composición y asociación resultan más adecuadas que la herencia.

Esto permite que los objetos colaboren entre sí manteniendo responsabilidades diferentes.

Decisión de no utilizar herencia

No se utilizó una jerarquía de herencia entre las clases principales porque no se identificó una relación clara de tipo “es un” que la justificara.

Por ejemplo:

- Memoria no es un Simulador
- Proceso no es una Memoria
- PlanificadorRoundRobin no es un Proceso

Crear una superclase común únicamente para reutilizar código habría generado una relación artificial.

Por este motivo se priorizaron interfaces y composición. La propia rúbrica establece que la herencia debe utilizarse cuando exista una relación “es un” justificable y que, si no se utiliza, debe poder explicarse la alternativa adoptada.

Decisión de no utilizar clases abstractas

Tampoco fue necesario incorporar clases abstractas.

Las clases del proyecto no comparten una implementación base suficientemente significativa como para justificar una superclase abstracta. En los casos donde era necesario establecer un contrato común se utilizaron interfaces.

Por ejemplo:

- Memoria implements IMemoria
- Proceso implements IProceso
- FirstFit implements IPoliticaAsignacion
- BestFit implements IPoliticaAsignacion
- WorstFit implements IPoliticaAsignacion
- PlanificadorRoundRobin implements IPlanificador
- Simulador implements ISimulador

Esto permite definir qué comportamiento debe ofrecer cada componente sin obligar a compartir una implementación que no corresponde al dominio.

La ausencia de clases abstractas es, por lo tanto, una decisión de diseño y no una omisión: para este modelo resultaron más adecuadas las interfaces y la composición.

Abstracción de la política de memoria

La política de asignación se separó mediante la interfaz:

IPoliticaAsignacion

Memoria no necesita que su atributo de política sea específicamente FirstFit, sino que trabaja contra ese contrato.

Sobre este contrato se implementaron tres estrategias: FirstFit, BestFit y WorstFit. Memoria recibe la política por constructor y el Simulador permite indicarla al configurarse, utilizando First Fit por defecto. De esta manera, incorporar una nueva estrategia solo requiere crear una clase que implemente IPoliticaAsignacion, sin modificar Memoria ni Simulador.

Uso de colecciones

Las relaciones de uno a muchos se modelaron mediante colecciones.

Por ejemplo:

- Simulador -> Proceso[]
- Memoria -> BloqueMemoria[]
- PlanificadorRoundRobin -> IProceso[]

Cada colección representa una relación concreta del dominio: los procesos registrados, los bloques que componen la memoria y la cola de procesos listos.

Las colecciones internas se mantienen encapsuladas y no se exponen directamente. En particular, el planificador devuelve únicamente el orden de los PID mediante `obtenerPids()`, evitando entregar referencias modificables a los procesos almacenados.

En conjunto, estas decisiones permitieron construir una solución donde los objetos poseen responsabilidades definidas, protegen su propio estado y colaboran mediante contratos claros.

INTERFACES, ABSTRACCIÓN, COMPOSICIÓN, ENCAPSULAMIENTO Y SOLID

La solución fue diseñada utilizando distintos mecanismos propios de la Programación Orientada a Objetos con el objetivo de mantener responsabilidades claras, proteger el estado interno y reducir el acoplamiento entre componentes. La consigna valora específicamente el uso de interfaces, abstracciones, composición, encapsulamiento y principios SOLID, además de la capacidad de justificar las decisiones adoptadas.

Interfaces

Se utilizaron interfaces para definir los contratos que deben respetar los principales componentes del simulador. Entre ellas se encuentran:

- ISimulador
- IProceso
- IMemoria
- IBloqueMemoria
- IPlanificador
- IPoliticaAsignacion
- IMetricas
- IEstadoSistema
- IVistaProceso
- IVistaBloqueMemoria

Estas interfaces permiten separar qué debe hacer un componente de cómo lo hace concretamente.

Por ejemplo:

FirstFit implements IPoliticaAsignacion

indica que FirstFit debe respetar el contrato definido para una política de asignación.

De forma similar:

- Memoria implements IMemoria
- Proceso implements IProceso
- PlanificadorRoundRobin implements IPlanificador
- Simulador implements ISimulador

Esta decisión facilita el reemplazo o ampliación de determinados componentes sin obligar a modificar todas las clases que los utilizan.

Abstracción

La abstracción se aplicó representando únicamente aquellos elementos del dominio que resultan necesarios para la simulación.

Por ejemplo, un proceso real de un sistema operativo posee mucha más información que la utilizada en este proyecto. Sin embargo, para resolver el escenario planteado se modelaron solamente datos relevantes como:

- PID
- memoria requerida
- tiempo total de CPU
- CPU restante
- estado
- quantum consumido
- tiempo de bloqueo

De la misma manera, la memoria se representa mediante bloques con:

- inicio
- tamaño
- PID asignado

sin intentar reproducir todos los detalles de la gestión de memoria de un sistema operativo real.

La política de asignación también se abstraio mediante `IPoliticaAsignacion`, de manera que Memoria necesita saber que existe una operación capaz de buscar un bloque adecuado, pero no necesita conocer todos los detalles internos de la estrategia utilizada.

Polimorfismo

El polimorfismo se aplica en la selección de la política de asignación. `FirstFit`, `BestFit` y `WorstFit` implementan la interfaz `IPoliticaAsignacion`, que define la operación `buscarBloque()`. Memoria invoca esa operación sin conocer qué clase concreta recibió, por lo que el mismo mensaje produce un comportamiento distinto según el objeto que lo responde.

Esto evita depender de condicionales por tipo concreto: Memoria no contiene un `if` o `switch` que pregunte qué política se está usando. La decisión queda resuelta por el objeto política inyectado en su constructor.

El polimorfismo se verifica en las pruebas automatizadas. El test "asigna distinto según la política sin modificar Memoria" ejecuta el mismo escenario con las tres políticas: con huecos libres de 200, 100 y 600 unidades en las direcciones 0, 250 y 400, una solicitud de 80 unidades se asigna en la dirección 0 con First Fit, en la 250 con Best Fit y en la 400 con Worst Fit. Además, el test "permite elegir la política de asignación" verifica el mismo comportamiento desde el Simulador.

Composición y colaboración entre objetos

La solución utiliza principalmente relaciones de tipo "tiene un", por lo que se priorizó la composición y colaboración entre objetos antes que la herencia.

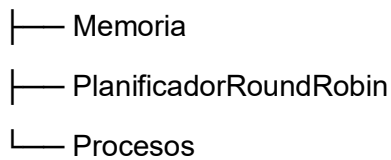
El Simulador mantiene y coordina:

- una memoria
- un planificador
- una colección de procesos
- como máximo un proceso en CPU

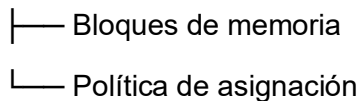
La clase Memoria, a su vez, administra una colección de BloqueMemoria y utiliza una política de asignación.

Conceptualmente:

Simulador



Memoria



Esto permite distribuir las tareas entre objetos especializados. Por ejemplo, Simulador coordina la ejecución, pero no decide directamente qué bloque debe asignarse; esa responsabilidad se delega a Memoria y a la política de asignación configurada.

La rúbrica solicita precisamente modelar correctamente las relaciones de composición y colaboración evitando acoplamientos innecesarios.

Encapsulamiento

El encapsulamiento fue una de las decisiones más importantes del diseño.

Las estructuras que representan el estado interno del sistema se mantienen privadas y sus modificaciones se realizan mediante métodos que aplican las reglas del dominio.

Por ejemplo, en Proceso no se modifica directamente el estado desde el exterior. Se utilizan operaciones como:

- marcarEsperandoMemoria()
- marcarListo()
- marcarEjecutando()
- marcarTerminado()
- bloquearPorEntradaSalida()

Cada una controla si la transición solicitada es válida.

También se protege la colección interna de bloques de Memoria, evitando que otro componente pueda reorganizarla directamente sin pasar por `asignarMemoria()` o `liberarMemoria()`.

Además, `Simulador` no devuelve directamente los objetos `Proceso` que utiliza internamente. En su lugar, construye objetos de consulta mediante `IVistaProceso` e `IEstadoSistema`. Esta decisión permite observar el sistema sin entregar referencias modificables a sus objetos internos, tal como exigen RF02 y RF10.

El mismo criterio fue aplicado al planificador. Su colección interna almacena procesos, pero el método de consulta:

`obtenerPids()`

devuelve únicamente el orden de sus identificadores, evitando exponer los objetos almacenados.

De esta manera se aplica el concepto de doble encapsulamiento, protegiendo tanto las colecciones como los objetos contenidos en ellas.

Aplicación de principios SOLID

S — Single Responsibility Principle

El principio de responsabilidad única se refleja en la separación de funciones entre las clases.

Por ejemplo:

- `Proceso` gestiona el estado y ejecución de un proceso.
- `Memoria` administra bloques y liberación.
- `FirstFit`, `BestFit` y `WorstFit` deciden qué bloque seleccionar.
- `PlanificadorRoundRobin` administra la cola de listos.
- `Simulador` coordina la interacción entre los componentes.

De esta forma, ninguna clase concentra por sí sola toda la lógica del sistema.

O — Open/Closed Principle

El diseño de la política de memoria permite extender el comportamiento sin modificar la clase `Memoria`. Partiendo de `FirstFit`, se incorporaron `BestFit` y `WorstFit` implementando el mismo contrato `IPoliticaAsignacion`, sin necesidad de reescribir la lógica de asignación, división o liberación de bloques de Memoria.

L — Liskov Substitution Principle

No se creó una jerarquía de herencia entre las clases principales, ya que no existía una relación clara de tipo “es un”.

Por esa razón, el principio de sustitución no se aplica mediante subclases tradicionales. El diseño evita introducir herencia artificial y utiliza interfaces cuando se necesita un contrato común.

Las implementaciones deben respetar los contratos de sus interfaces para poder ser utilizadas allí donde se espera dicha abstracción. En el caso de las políticas de memoria, FirstFit cumple con IPoliticaAsignacion. FirstFit, BestFit y WorstFit son sustituibles entre sí allí donde se espera una IPoliticaAsignacion, y las pruebas verifican que Memoria funciona correctamente con cualquiera de las tres.

I — Interface Segregation Principle

Se evitó crear una única interfaz grande que contuviera todas las operaciones del sistema.

En su lugar se definieron contratos específicos:

- IMemoria
- IProceso
- IPlanificador
- IPoliticaAsignacion
- ISimulador

y otras interfaces destinadas únicamente a consulta y representación de datos.

De esta forma, una clase implementa únicamente las operaciones relacionadas con su responsabilidad.

D — Dependency Inversion Principle

Este principio se aplica principalmente mediante el uso de abstracciones.

Por ejemplo, Memoria trabaja con:

IPoliticaAsignacion

en lugar de depender exclusivamente de FirstFit.

Del mismo modo, dentro de Simulador existen referencias tipadas mediante abstracciones como:

- IMemoria
- IPlanificador
- IProceso

Esto reduce la dependencia directa respecto de implementaciones concretas.

La política de asignación se inyecta desde el exterior: el Simulador la recibe por constructor y la entrega a Memoria, que solo conoce la abstracción. La aplicación no es absoluta, ya que el Simulador continúa creando las instancias concretas de Memoria y

PlanificadorRoundRobin, por lo que puede considerarse una aplicación parcial del principio.

POLÍTICAS DE ASIGNACIÓN DE MEMORIA

La memoria se administra mediante asignación contigua. Para elegir el bloque se implementaron tres políticas que cumplen la interfaz `IPoliticaAsignacion`: First Fit, que selecciona el primer bloque libre suficiente; Best Fit, que selecciona el bloque libre suficiente de menor tamaño; y Worst Fit, que selecciona el de mayor tamaño. Ante empates se elige el bloque de menor dirección. La política se indica al crear el Simulador y, si no se especifica, se utiliza First Fit.

La política solo decide qué bloque usar; Memoria es quien modifica la estructura. Si el bloque elegido es mayor al solicitado, se divide en una parte ocupada y otra libre. Si coincide exactamente, se asigna completo sin generar bloques de tamaño cero. Si no existe un bloque suficiente, la asignación falla sin alterar la memoria.

Antes: [Libre: 600] → Después: [Proceso: 200] [Libre: 400]

Cuando un proceso termina, su bloque se libera y se realiza la coalescencia: los bloques libres adyacentes se fusionan en uno solo, sin desplazar procesos ni compactar la memoria.

[Libre: 200] [Libre: 300] [Proceso: 100] → [Libre: 500] [Proceso: 100]

Comparación de las políticas

Se utilizó un mismo escenario para las tres: una memoria de 1000 unidades con huecos libres de 200 (dirección 0), 100 (dirección 250) y 600 (dirección 400), y una solicitud de 80 unidades. Este caso está verificado en las pruebas automatizadas.

Política	Dirección asignada	Sobrante del hueco
First Fit	0	120
Best Fit	250	20
Worst Fit	400	520

First Fit es la más simple, ya que puede detener la búsqueda en el primer bloque válido. Best Fit preserva los huecos grandes, pero deja sobrantes pequeños difíciles de reutilizar. Worst Fit busca que el sobrante siga siendo útil, pero consume primero los bloques grandes. Best Fit y Worst Fit deben recorrer todos los bloques para decidir. Ninguna elimina la fragmentación externa; solo cambia cómo se distribuyen los huecos resultantes.

ESTADOS DE LOS PROCESOS Y ORDEN DE EJECUCIÓN DE CADA TICK

El simulador utiliza seis estados para representar el ciclo de vida de los procesos: NUEVO, ESPERANDO_MEMORIA, LISTO, EJECUTANDO, BLOQUEADO y TERMINADO. Un proceso comienza en NUEVO; si consigue memoria pasa a LISTO, y si no consigue queda en ESPERANDO_MEMORIA hasta que pueda ser admitido. Cuando obtiene la CPU pasa a EJECUTANDO. Si se produce una operación de Entrada/Salida pasa a BLOQUEADO, conserva la memoria y deja de consumir CPU hasta que finaliza su tiempo de bloqueo, momento en el que vuelve a LISTO. Finalmente, cuando su CPU restante llega a cero pasa a TERMINADO y libera la memoria asignada.

Cada llamada a `avanzarTick()` representa una unidad de tiempo y sigue siempre el mismo orden. Primero se intenta admitir a los procesos nuevos o en espera de memoria; después se actualizan los procesos bloqueados; luego se realiza el despacho y ejecución mediante Round Robin; y al final se incrementa el tick y quedan actualizadas las métricas. Como máximo un proceso consume una unidad de CPU por tick. Si un proceso termina, libera su memoria en ese mismo tick. Si se dispara una Entrada/Salida, el bloqueo tiene prioridad sobre la rotación por quantum. Si el quantum se agota y hay otros procesos listos, el proceso actual vuelve al final de la cola; si no hay otros, continúa ejecutándose con el quantum renovado.

Matriz RF01–RF10 → clases/métodos → tests

RF	Clases y métodos principales	Tests representativos
RF01 – Configurar e iniciar la simulación	Simulador.constructor, Memoria.constructor()	inicia con la configuración indicada, rechaza un quantum inválido, rechaza una memoria inválida, inicia con un único bloque libre del tamaño total. Permite elegir la política de asignación
RF02 – Registrar y consultar procesos	Simulador.registrarProceso(), obtenerProcesos(), Proceso.constructor()	registra y consulta procesos, rechaza un PID duplicado, rechaza datos inválidos al registrar procesos, protege los procesos de modificaciones externas.
RF03 – Gestionar estados y admisión	Simulador.avanzarTick(), Proceso.marcarEsperandoMemoria(), marcarListo()	admite y ejecuta un proceso cuando consigue memoria, deja esperando memoria a un proceso que no encuentra espacio, admite otro proceso aunque uno anterior no entre en memoria
RF04 – Asignar memoria contigua	Memoria.asignarMemoria(), FirstFit.buscarBloque(), BestFit.buscarBloque(), WorstFit.buscarBloque()	elige el primer bloque libre suficiente, elige el bloque libre suficiente de menor tamaño, elige el bloque libre suficiente de mayor tamaño, ante un empate elige el bloque de menor dirección, asigna distinto según la política sin modificar Memoria, divide un bloque cuando sobra espacio, realiza una asignación exacta sin crear un bloque vacío, no modifica la memoria si no existe un bloque suficiente

RF05 – Liberar memoria y realizar coalescencia	Memoria.liberarMemoria(), fusionarBloquesLibres()	fusiona con el bloque libre de la izquierda, fusiona con el bloque libre de la derecha, fusiona bloques libres a ambos lados, finaliza un proceso y libera su memoria en el mismo tick
RF06 – Avanzar un tick de forma determinista	Simulador.avanzarTick(), Proceso.ejecutarTick()	permite que solo un proceso consuma CPU por tick, admite en el siguiente tick después de liberar memoria, mantiene los bloques de memoria sin solapamientos
RF07 – Planificar CPU con Round Robin	PlanificadorRoundRobin.encolar(), desencolar(), Simulador.avanzarTick(), Proceso.reiniciarQuantum()	rota al proceso cuando agota el quantum y hay otro listo, renueva el quantum si no hay otro proceso listo, finaliza en el límite del quantum sin volver a la cola, respeta la secuencia completa de round robin
RF08 – Simular Entrada/Salida	Proceso.configurarEntradaSalida(), debeBloquearsePorEntradaSalida(), bloquearPorEntradaSalida(), actualizarBloqueo()	se bloquea al alcanzar el momento de entrada y salida, vuelve a listo cuando termina el bloqueo, no consume CPU mientras está bloqueado y vuelve a ejecutar, da prioridad al bloqueo sobre la rotación por quantum
RF09 – Exponer métricas	Simulador.obtenerMetricas()	inicia las métricas con los valores correctos, calcula la fragmentación externa de la memoria, calcula las métricas cuando la memoria está completamente ocupada, calcula la utilización de cpu con ticks ocupados y ociosos, tests de cambios de contexto

RF10 – Consultar el estado del sistema	Simulador.obtenerEstado(), crearVistaProceso(), PlanificadorRoundRobin.obtenerPids()	consulta el proceso en cpu la cola de listos y el mapa de memoria, consulta procesos bloqueados esperando memoria y terminados, no duplica procesos entre los estados del sistema, protege los procesos de modificaciones externas
--	--	---

RESULTADOS DE LAS PRUEBAS Y COBERTURA

La solución se verificó mediante pruebas automatizadas con Vitest, incluyendo casos normales, casos límite y validaciones de error. En la última ejecución se obtuvieron 69 tests aprobados, distribuidos en 8 archivos de prueba, sin fallos.

Los tests cubren la configuración inicial, el registro y la consulta de procesos, la asignación y liberación de memoria con las tres políticas (First Fit, Best Fit y Worst Fit), la coalescencia, la planificación Round Robin, los eventos de Entrada/Salida, los cambios de contexto, las métricas y los invariantes del sistema.

La cobertura se midió con el proveedor V8 de Vitest (@vitest/coverage-v8) sobre el código de producción de la carpeta src/. Para reproducirla:

```
npm ci
npm run coverage
```

Métrica	Cobertura
Statements	97,76 %
Branches	95,03 %
Functions	100 %
Lines	97,71 %

La cobertura de líneas alcanzó 97,71 %, superando el 90 % requerido. Las tres políticas de asignación, Memoria, BloqueMemoria y PlanificadorRoundRobin alcanzaron el 100 % de cobertura.

Las únicas líneas sin cubrir son validaciones defensivas que el flujo normal del simulador no puede alcanzar:

- En Proceso, los rechazos de transiciones de estado que el Simulador nunca solicita y los retornos anticipados de los métodos de bloqueo.
- En Simulador, el error que se lanza si un proceso de la cola de listos no está registrado. Ese caso no puede darse, porque solo se encolan procesos registrados.

Estas validaciones se mantienen para proteger los invariantes del dominio ante un uso incorrecto de las clases.

GitHub Actions ejecuta automáticamente, en cada push o pull request sobre main, la instalación de dependencias, la verificación de TypeScript, los tests y la medición de cobertura. El tag evaluado, v1.0-ae2, finalizó correctamente en integración continua.

```
PS C:\Users\saque\OneDrive\Escritorio\Simulador-Procesos> npm test
✓ tests/Proceso.test.ts (11 tests) 15ms
✓ tests/Memoria.test.ts (11 tests) 18ms
✓ tests/Simulador.test.ts (34 tests) 34ms
✓ tests/BestFit.test.ts (3 tests) 4ms

Test Files  8 passed (8)
Tests       69 passed (69)
Start at    22:43:53
Duration    1.17s (transform 69%, import 24%, tests 4%, worker 3%)
```

Figura 1: Ejecución de la suite de pruebas con Vitest.

```
PS C:\Users\saque\OneDrive\Escritorio\Simulador-Procesos> npm run coverage
```

All files	97.76	95.03	100	97.71	
BestFit.ts	100	100	100	100	
BloqueMemoria.ts	100	100	100	100	
EstadoProceso.ts	100	100	100	100	
FirstFit.ts	100	100	100	100	
Memoria.ts	100	100	100	100	
PlanificadorRoundRobin.ts	100	100	100	100	
Proceso.ts	93.44	88.88	100	93.44	56,70,163,176
Simulador.ts	98.87	97.77	100	98.85	155
WorstFit.ts	100	100	100	100	

Figura 2: Reporte de cobertura de líneas.

The screenshot shows the GitHub Actions interface for the repository 'sophisaquer / Simulador-Procesos'. The 'Actions' tab is selected, displaying a list of workflow runs. The left sidebar shows the 'tests' workflow selected. The main area shows 12 workflow runs, with the first four visible:

- completo la documentación de las políticas** (tests #12: Commit beb6ade pushed by sophisaquer) - Status: main - Completed 7 minutes ago (13s)
- documento las políticas de asignación** (tests #11: Commit 635b59a pushed by sophisaquer) - Status: main - Completed 30 minutes ago (12s)
- protejo la cola del planificador** (tests #10: Commit 60d9331 pushed by sophisaquer) - Status: main - Completed Oct 2, 12:21 PM GMT-3 (20s)
- ajusto las pruebas de coalescencia** (tests #9: Commit a34c27e pushed by sophisaquer) - Status: main - Completed Oct 2, 12:02 PM GMT-3 (12s)

Figura 3: Ejecuciones del workflow de GitHub Actions.

ANÁLISIS DE FRAGMENTACIÓN EXTERNA

La **fragmentación externa** aparece cuando existe memoria libre suficiente en total, pero se encuentra dividida en varios bloques no contiguos. En esta situación, un proceso puede no ser admitido aunque la suma de la memoria disponible alcance, debido a que la asignación utilizada es contigua y necesita encontrar un único bloque de tamaño suficiente.

El simulador calcula la fragmentación externa mediante:

$$\text{Fragmentación externa} = \left(1 - \frac{\text{mayor bloque libre}}{\text{memoria libre total}}\right) \times 100$$

Por ejemplo, en uno de los escenarios de prueba se obtienen dos huecos libres de 100 y 300 unidades. La memoria libre total es entonces 400 y el mayor bloque disponible es de 300:

$$\left(1 - \frac{300}{400}\right) \times 100 = 25\%$$

Por lo tanto, la fragmentación externa resultante es del 25%. Este caso se encuentra verificado mediante pruebas automatizadas.

Las tres políticas pueden producir este tipo de fragmentación, ya que asignan bloques sin reorganizar los ocupados. Con sucesivas asignaciones y liberaciones pueden quedar pequeños huecos separados entre sí.

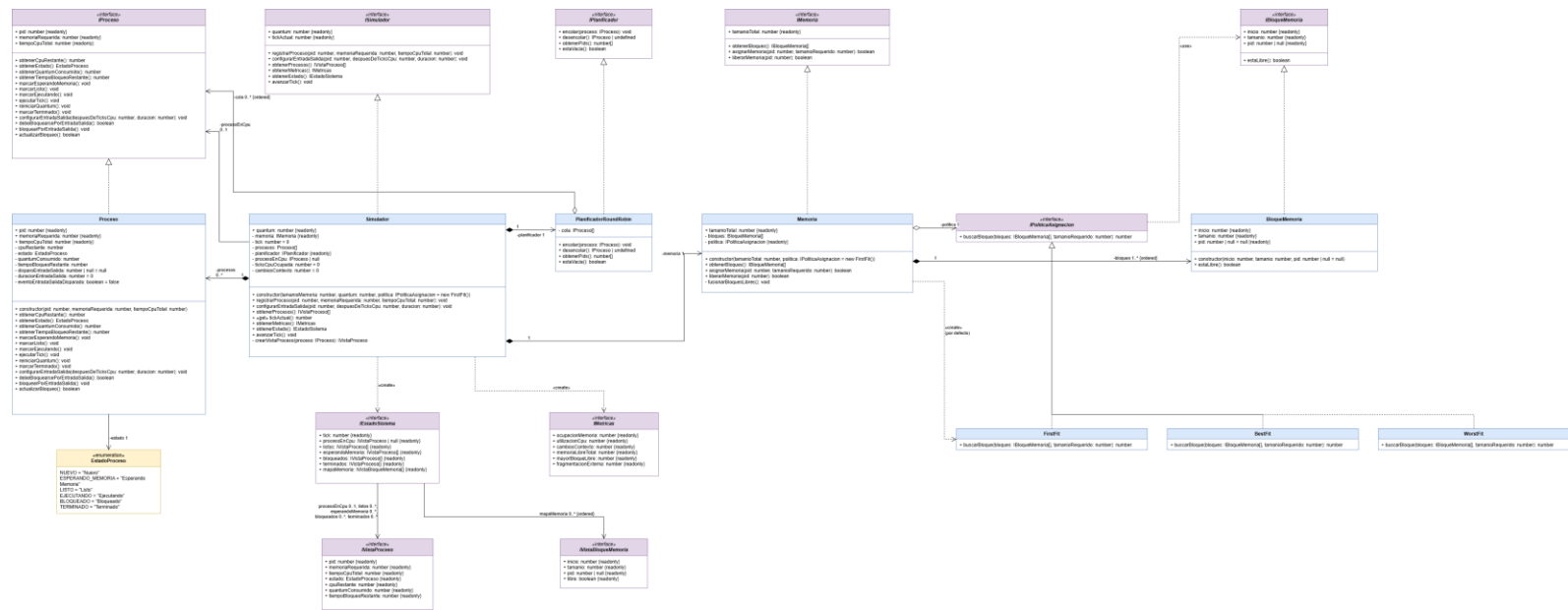
Para disminuir esta fragmentación, cuando un proceso termina la clase Memoria realiza coalescencia, fusionando automáticamente los bloques libres que se encuentran contiguos. Por ejemplo:

[Libre 100] [Libre 300] → [Libre 400]

La coalescencia no desplaza procesos ni realiza compactación; solamente une espacios libres adyacentes. De esta manera aumenta el tamaño de los huecos disponibles y mejora la posibilidad de admitir procesos posteriores.

Cuando toda la memoria está ocupada, la memoria libre total es cero y el simulador establece la fragmentación externa en 0%, evitando una división por cero. Este comportamiento también fue incluido dentro de las pruebas del sistema.

DIAGRAMA DE CLASES

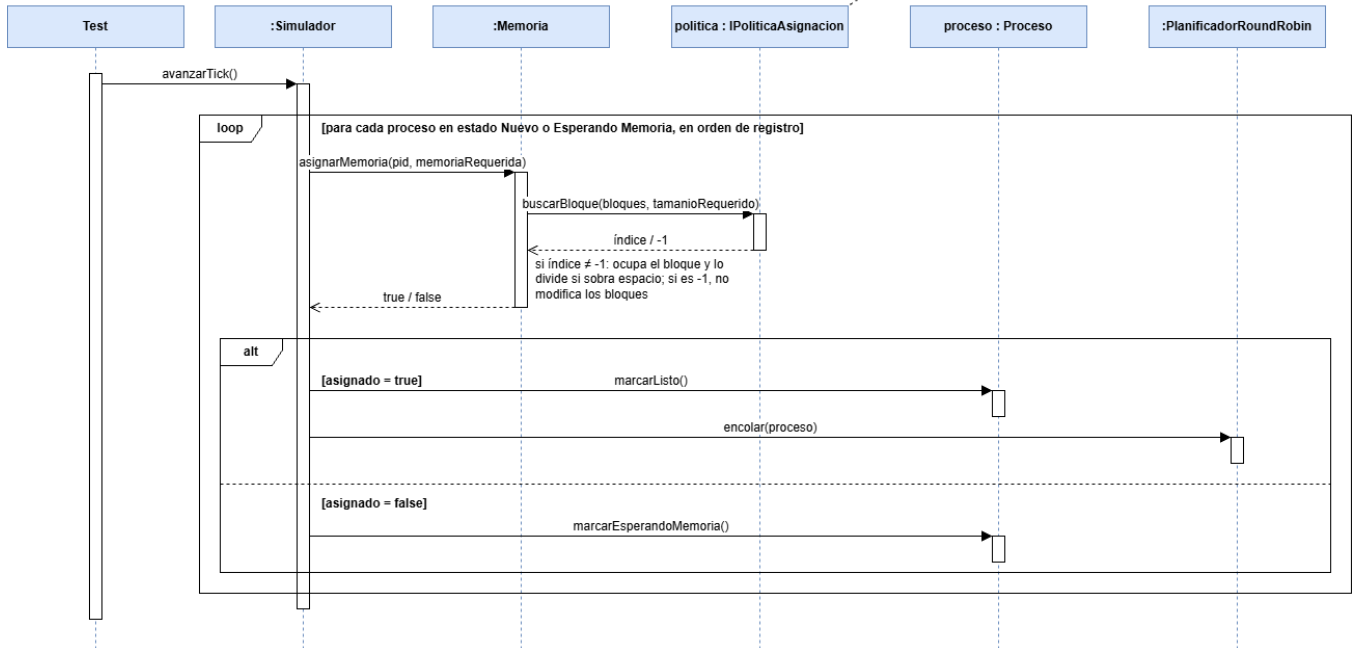


La versión en alta resolución y el archivo editable se encuentran en la carpeta docs/ del repositorio.

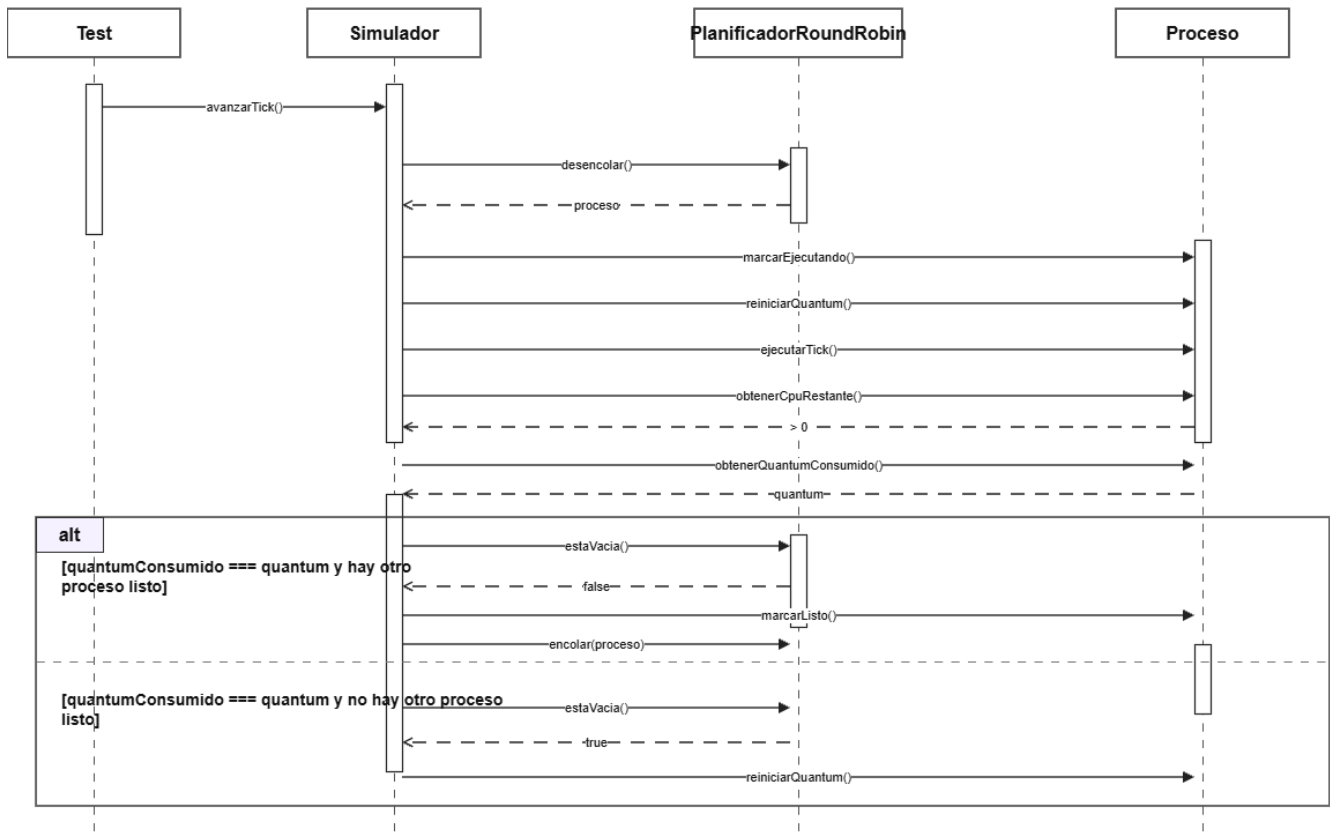
DIAGRAMAS DE SECUENCIA

Admisión y asignación de memoria – RF03 / RF04

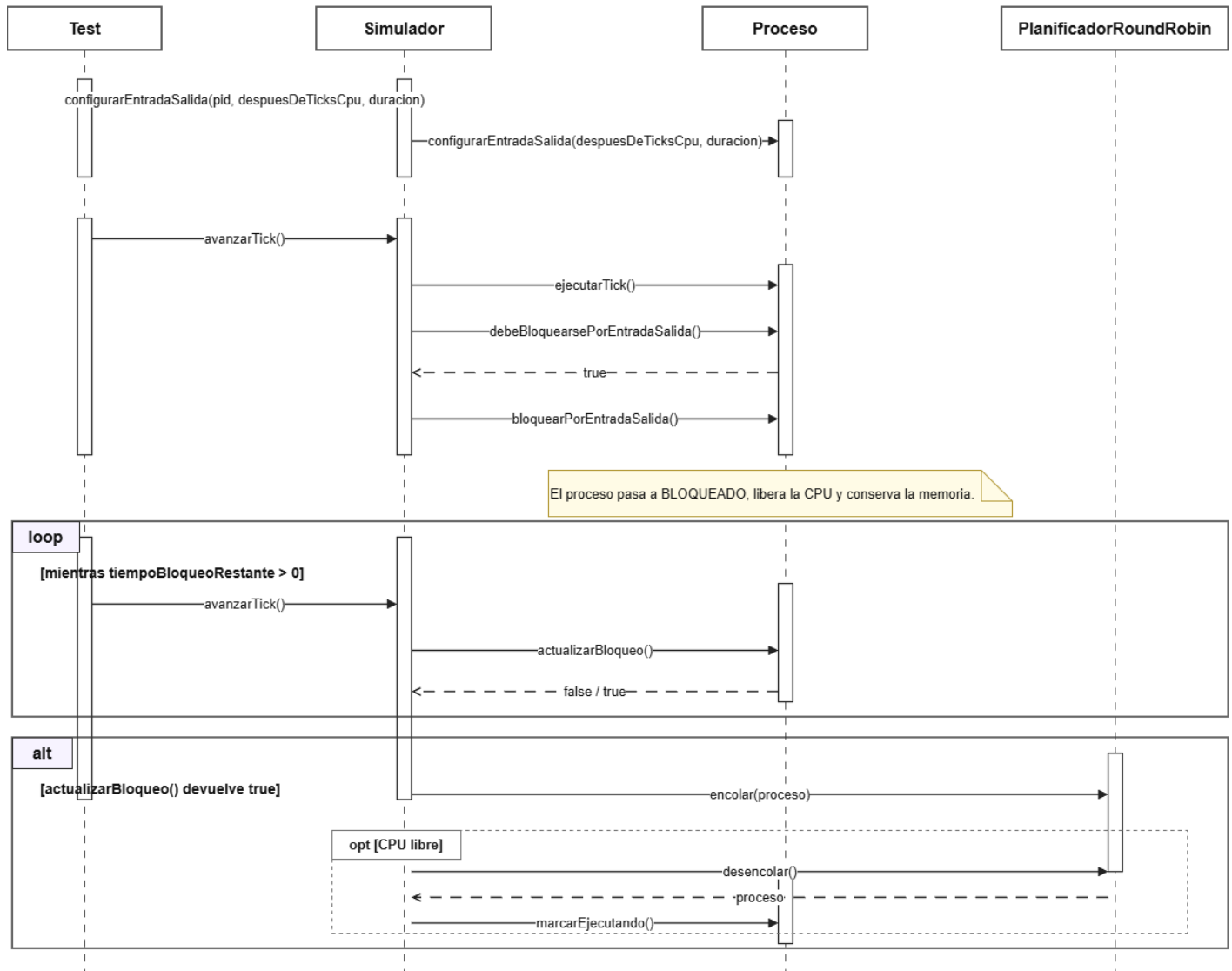
FirstFit, BestFit o WorstFit, según la política indicada al crear el Simulador (First Fit por defecto). Memoria solo conoce la interfaz.



Tick de Round Robin – RF06/RF07



Bloqueo y retorno por Entrada/Salida – RF08



CONCLUSIONES

El desarrollo del simulador permitió integrar conceptos de Programación Orientada a Objetos con un problema propio de Sistemas Operativos, modelando procesos, memoria, planificación de CPU, eventos de Entrada/Salida y métricas dentro de una misma solución.

La organización mediante clases con responsabilidades específicas permitió evitar una concentración excesiva de lógica en Simulador. Componentes como Proceso, Memoria, FirstFit y PlanificadorRoundRobin mantienen comportamientos bien definidos y colaboran entre sí mediante interfaces y composición. El encapsulamiento del estado interno y el uso de vistas de consulta también permitieron proteger las reglas del dominio frente a modificaciones externas.

Las políticas First Fit, Best Fit y Worst Fit permitieron resolver la asignación contigua de memoria de forma intercambiable, mientras que la liberación y coalescencia de bloques permitió analizar situaciones de fragmentación externa. Por su parte, la planificación Round Robin permitió representar el reparto de CPU mediante quantum, rotación de procesos y cambios de contexto, incorporando además bloqueos por Entrada/Salida y retorno posterior a la cola de listos.

Las pruebas automatizadas permitieron verificar el comportamiento de los distintos componentes y detectar errores durante el desarrollo. La solución final alcanzó una cobertura global de líneas del 97,71%, con 69 tests aprobados, complementados con integración continua mediante GitHub Actions.

En conjunto, el proyecto permitió construir una solución modular, verificable y coherente con los requerimientos definidos, además de reforzar conceptos como encapsulamiento, abstracción, composición, uso de interfaces, separación de responsabilidades y validación automática del software.

FUENTES CONSULTADAS

Microsoft. (s. f.). *TypeScript Documentation*. TypeScript. [TypeScript](#)

Vitest. (s. f.). *Coverage*. Vitest Guide. La documentación explica el uso de cobertura con V8 e Istanbul y la configuración correspondiente. [Vitest](#)

GitHub. (s. f.). *GitHub Actions documentation*. GitHub Docs. [GitHub Docs](#)

OpenJS Foundation. (s. f.). *Node.js Documentation*. Node.js. [Node.js](#)

Universidad de la Cuenca del Plata. (2026). *Actividad de Evaluación 2 – Paradigmas y Lenguajes de Programación II*. Material de cátedra.

Universidad de la Cuenca del Plata. (2026). *Actividad de Evaluación 2 – Sistemas Operativos*. Material de cátedra.